# DD2525 Project Report

## GraphQL Security

Authors: Tomás Sobral Teixeira

Eva Sophia Schreyer

# Contents


**1 Introduction** **3**

1.1 Background . . . . . 3

1.2 Goal . . . . . 4

**2 Reconnaissance** **5**

**3 SQL Injection** **7**

3.1 Concept . . . . . 7

3.2 Code Documentation . . . . . 7

3.3 Results . . . . . 8

**4 DoS Batching** **10**

4.1 Concept . . . . . 10

4.2 Code Documentation . . . . . 10

4.3 Results . . . . . 12

4.4 Mitigation . . . . . 14

**5 DoS Recursion** **15**

5.1 Concept . . . . . 15

5.2 Code Documentation . . . . . 16

5.3 Results . . . . . 17

5.4 Mitigation . . . . . 20

# 1 Introduction

## 1.1 Background

GraphQL is a **query language** for APIs paired with a server-side execution runtime, i.e. when a query arrives, the runtime parses and validates it against the schema, invokes the appropriate data resolvers (e.g., database or backend service calls), and assembles the JSON response exactly as requested by the client. Developed internally by Facebook in 2012 and released publicly in 2015, GraphQL was designed to overcome limitations of traditional REST architectures – particularly in scenarios where clients need complex, nested, or highly specific data – by enabling a single request to one endpoint that returns precisely the fields and relationships needed.

In a REST-based system, clients often make multiple calls to different endpoints and risk over-fetching or under-fetching data. In contrast, GraphQL exposes a **single endpoint** where clients can send one request that explicitly specifies exactly which fields and relationships they need. This precision minimizes unnecessary data transfer and simplifies client–server interactions.

A GraphQL request resembles a JSON-like query structure. For example:

```
query { user(id: "1") { name email posts { title } } }
```

GraphQL operates on a strongly-typed schema that defines all available types, queries (read operations), mutations (write operations), and subscriptions (real-time updates). This schema is **introspectable**, meaning clients can query the API itself to discover all available types, fields, and operations at runtime.

While this flexibility offers significant advantages, it also introduces new security considerations. Common vulnerabilities in GraphQL APIs include:

- **Injection attacks** (e.g., SQL injection, NoSQL injection) through unvalidated inputs.
- **Information disclosure** via unrestricted introspection, allowing attackers to map out the entire schema.
- **Denial-of-Service (DoS)** through excessively complex, deeply nested, or batched queries that exhaust server resources.

## 1.2 Goal

The goal of this project is to uncover and evaluate security weaknesses in GraphQL APIs by building and applying specialized fuzzing tools. These tools focus on vulnerability classes unique to GraphQL – particularly injection and denial-of-service (DoS) attacks – and leverage schema introspection to automate testing wherever possible. We validate their effectiveness against several open-source GraphQL projects (run locally) and one public endpoint (that specifically allows experimentation), assessing how well these codebases withstand real-world adversarial inputs.

To achieve this, the project introduces and implements three distinct fuzzers:

- **GraphQL SQLi Fuzzer** – Automates the injection of malicious SQL payloads into GraphQL queries by targeting all string-based input arguments. It leverages introspection to discover available query structures and recursively crafts valid queries with embedded SQL payloads, testing for classic SQL injection vulnerabilities.
- **DoS Batching Fuzzer** – Exploits the batching feature of GraphQL, where multiple operations can be sent in a single request. This fuzzer uses introspection to discover available queries, identifies the slowest query and then automatically generates and sends a batch of that query. It thus attempts to overload the server's request processing capability, potentially causing latency spikes or service disruption.
- **DoS Recursive Fuzzer** – Crafts deeply nested queries for fields that return object types, aiming to exhaust CPU, memory, or stack resources. Because automated detection of recursive field chains proved impractical, this tool requires manual specification of recursion points from the schema. Once provided, it iteratively increases nesting depth to reveal the server's maximum safe recursion limit.

We deployed and tested these fuzzers on five large, open-source GraphQL projects (hosted on GitHub) plus one publicly accessible GraphQL API that explicitly permits testing. By evaluating fuzzing results across diverse schema designs, data models, and server configurations, we gauge the robustness and generalizability of our approach and identify common patterns of misconfiguration or missing defenses.

# 2 Reconnaissance

Reconnaissance is the essential first phase of any GraphQL security assessment. It encompasses two main tasks: locating the GraphQL endpoint and enumerating its schema. Many high-quality tools exist for these purposes, and we briefly summarize the main ones below.

### Endpoint Discovery

- *Graphw00f* (`https://github.com/dolevf/graphw00f`)
  A dedicated fingerprinting tool that detects GraphQL endpoints by analyzing HTTP responses for characteristic GraphQL signatures.
- *Content-Discovery via Kiterunner* (`https://github.com/assetnote/kiterunner`)
  Combined with a GraphQL-specific wordlist (e.g. `https://github.com/danielmiessler/SecLists/blob/master/Discovery/Web-Content/graphql.txt`), this approach brute-forces common endpoint paths (e.g. '/graphql', '/api/graphql').
- *Passive Inspection with Burp Suite*
  Monitoring live traffic can reveal GraphQL requests embedded in application code (though sometimes they are hidden). Burp's *InQL* extension further aids in both endpoint identification and exploratory querying.

### Introspection

GraphQL exposes a standardized introspection query which is universal, i.e. when introspection is allowed, it returns the server's entire schema (types, fields, arguments, etc.). There are only some cases where the general introspection does not return everything, e.g. for custom metadata or to omit deprecated or internal types.
The result of the introspection, called schema, is different for every GraphQL server and its underlying database.
Although some server implementations customize or disable introspection, many still leak schema details either through error messages or non-standard hints.

- *Standard Introspection*
  Execute the generalized introspection query to obtain the complete Schema Definition Language (SDL) representation, which is essentially the blueprint of the GraphQL API.
- *Enumeration when Introspection is Disabled*
  Even with disabled introspection, many servers still leak the schema simply by returning error messages with field suggestions. For this case, we did not find a good working tool; however, the schema can be inferred manually (even though this is very time consuming).

### Schema Visualization

After getting the introspection result, we can load it into a visualization tool (e.g. `https://graphql-kit.com/graphql-voyager/`) to see all types, fields, relationships, and nested structures. This interactive view greatly speeds up the identification of high-risk operations (deeply nested, large collections, or recursive types).

### Interactive Exploration with GraphiQL

Many GraphQL servers expose an in-browser IDE such as GraphiQL or GraphQL Playground. Although this is not intended for production and thus typically not exposed to the web, they allow testers to craft and execute queries in real time revealing valid field names, argument types, and sample responses.

# 3 SQL Injection

## 3.1 Concept

The GraphQL SQL Injection (SQLi) Fuzzer is designed to test GraphQL APIs for vulnerabilities caused by **improper input sanitization of string fields**. GraphQL's flexible query syntax and dynamic schema make it susceptible to traditional injection attacks when not properly secured, particularly through string arguments in queries.

The conceptual approach behind this fuzzer is as follows:

- **Schema Introspection:** The fuzzer begins by sending an introspection query to retrieve the GraphQL schema. It identifies all available query operations and their arguments.
- **Target Selection:** It selects queries that accept arguments of type String, as these are the most likely vectors for SQL injection.
- **Payload Injection:** For each discovered target, the fuzzer injects a list of well-known SQLi payloads into the string arguments.
- **Query Construction:** It builds GraphQL queries with the payloads.
- **Response Analysis:** The tool sends the queries and prints out the responses for signs of potential vulnerabilities such as database errors, timing anomalies, or changes in the structure of returned data.

## 3.2 Code Documentation

The SQLi fuzzer is implemented in a single file: `sqli.py`. Below is a breakdown of its structure and functionality:

- **get_named_type(type_info):** Recursively extracts the base GraphQL type name, skipping wrappers like NON_NULL and LIST.
- **introspect_schema(endpoint, headers):** Sends an introspection query to the GraphQL endpoint and returns a list of top-level query fields, their arguments, and return types.
- **get_type_fields(type_name, endpoint, headers):** Retrieves field names of a specific object type. It is used for generating valid sub-selections in queries with object return types.
- **generate_sql_payloads():** Returns a list of known SQLi payloads.

- **main():** Uses all the other functions to send meaningful queries with SQLi payloads and prints the responses to the standard output.

## 3.3 Results

We tested the SQL injection fuzzer in various open-source projects, more specifically in: DVGA, Vendure, Saleor, Keystone, and Strapi. All the outputs of running the fuzzer against these websites are located in the folder `SQL_Injection/`. The results show that most of these projects (except for DVGA) are protected against SQLi attacks since we get the following messages as responses to our queries (Saleor, Vendure, and Strapi, respectively):

- `To access this path, you need one of the following permissions: MANAGE_PRODUCTS, MANAGE_ORDERS, MANAGE_SHIPPING`
- `You are not currently authorized to perform this action`
- `Forbidden access`

These show that these projects implement robust access control mechanisms to protect their GraphQL APIs from unauthorized access. They use a combination of authentication (verifying who the user is) and authorization (verifying what the user is allowed to do). Authentication is typically handled through tokens such as JWTs, which must be included in the Authorization header of GraphQL requests. Once a user is authenticated, the system checks their role-based permissions to determine whether they are allowed to access specific queries or mutations. Each protected GraphQL resolver is configured to allow access only to users with specific permissions, such as MANAGE_PRODUCTS or MANAGE_ORDERS. If a user lacks the required permissions or is unauthenticated, the API returns clear error messages like "Forbidden access" or "You are not currently authorized to perform this action." These protections effectively block unauthorized users and automated fuzzers from probing or exploiting sensitive parts of the API.

On the other hand, we were able to find vulnerabilities in DVGA. When we ran the fuzzer against its endpoint, we got the folowing outputs:

```
message":"(sqlite3.OperationalError) near \"’’\": syntax error\n[SQL:
SELECT pastes.id AS pastes_id, pastes.title AS pastes_title,
pastes.content AS pastes_content, pastes.public AS pastes_public,
pastes.user_agent AS pastes_user_agent, pastes.ip_addr AS pastes_ip_addr,
pastes.owner_id AS pastes_owner_id, pastes.burn AS pastes_burn \nFROM
pastes \nWHERE pastes.public = 0 AND pastes.burn = 0 AND title = ’ ’’ or
content = ’ ’’ ORDER BY pastes.id DESC\n LIMIT ? OFFSET ?
```

{"data":{"pastes":[{"id":"12","title":"TITLE!!!!!"},
{"id":"11","title":"TITLE!!!!!"},{"id":"10","title":"Testing Testing"},
{"id":"9","title":"This is my first paste"},{"id":"8","title":"TITLE!!!!!"},
{"id":"7","title":"This is my first paste"},{"id":"6","title":"What is this
even"},{"id":"5","title":"What is this even"},{"id":"4","title":"This is my
first paste"},{"id":"3","title":"Testing Testing"},{"id":"2","title":"555-555-
1337"},{"id":"1","title":"Testing Testing"}]}}

The observed outputs provide strong evidence of a successful SQLi attack. Initially, the application returned a raw SQL error – `(sqlite3.OperationalError) near """: syntax error` – which indicates that the user input is being inserted directly into SQL queries without proper sanitization. This kind of error typically only appears when input is improperly escaped or not validated. Furthermore, after submitting a specially crafted payload, the response included database records, suggesting that the injected input was able to manipulate the underlying query logic to bypass filters. This combination of error messages and anomalous data retrieval clearly demonstrates that arbitrary SQL statements can influence the application's database behavior, confirming the presence of an SQL injection vulnerability. This would not happen if, like the other projects, there was some kind of authentication requirement or even simpler, sanitization of user input before integrating it into SQL queries.

Lastly, Keystone did not give any results since none of its query fields are of type string, the one being fuzzed for in this attack.

**Overall Takeaway**

With these results, we came to the conclusion that Strapi, Vendure, and Saleor successfully enforced robust access controls and input validation mechanisms that prevented unauthorized access and SQL injection attempts while DVGA remained susceptible to such attacks. This contrast highlights the importance of secure resolver logic, strict schema validation, and proper use of parameterized queries within GraphQL servers. It also shows how security in GraphQL is not provided by the query language itself but by how it is implemented. These findings reinforce the need for developers to adopt secure coding practices and perform regular security testing.

# 4 DoS Batching

## 4.1 Concept

Batching in GraphQL refers to the ability to send multiple operations (queries or mutations) within a single HTTP request. This feature is intended to reduce network overhead and enable parallel data fetching, thereby improving client-side performance. However, it also introduces a potential attack: a malicious client can exploit batching to overwhelm the server by *repeatedly sending resource-intensive queries in a single request*, leading to a Denial of Service (DoS) condition.

To automate the detection of this vulnerability, we developed a script that performs the following steps:

- **Schema Introspection:** The script initiates an introspection query to retrieve the full GraphQL schema from the target endpoint. It then extracts all available query operations along with their arguments.
- **Target Selection:** Using the discovered queries, the script sends individual requests and measures the response time for each. This allows it to identify the operation that imposes the highest computational cost on the server.
- **Batch Construction:** Once the most time-consuming query is identified, the script constructs a batched request containing multiple instances of that same query.
- **Response Analysis:** If the server does not support batching, it typically returns an error message indicating this (e.g., "batching not supported"). If batching is enabled, the server processes the batched request, and a noticeably increased response time indicates potential susceptibility to a DoS attack via batching.

This automated approach allows us to efficiently test GraphQL endpoints for batching-based DoS vulnerabilities with minimal manual intervention.

## 4.2 Code Documentation

The DoS batching functionality is implemented in `DoSBatching.py`. The script accepts the following optional command-line arguments (otherwise it uses defaults):

- **URL of the GraphQL endpoint** (`-u` or `--url`)

- **Introspection query JSON file** (`-i` or `--introspect`)
  Typically you can rely on the default introspection query (see Section 2), but for our testing we used individual introspection files to ensure that we captured everything.

- **Schema JSON file** (`-s` or `--schema`)
  If provided, the script skips introspection and uses this schema instead. This is useful when introspection is disabled and one has to get the schema through other methods (though this did not occur in our tests).

- **Batch size** (`-b` or `--batch`)
  Number of identical queries to include in the batched request.

The functions in `DoSBatching.py` are:

- **`load_introspection_query(path: str)` → `dict`:** Loads the JSON file containing the introspection query payload.

- **`load_schema_file(path: str)` → `dict`:** Loads a saved schema JSON, if provided via `-s`.

- **`send_query(url: str, payload: dict)` → `(status_code, elapsed_time)`:** Sends a POST request with the given payload to the GraphQL endpoint and measures the response time.

- **`extract_query_fields(schema: dict)` → `list`:** This function extracts all top-level query fields from a GraphQL schema and determines the minimal required subfields for each. First, it locates the top-level *Query* type within the schema's type definitions. Then, for each field under *Query*, it unwraps any nested type layers (e.g., NON_NULL, LIST) to get the core type. If the field returns a complex type (such as an OBJECT, INTERFACE, or UNION), then GraphQL requires at least one subfield selection. In that case, this function assigns "__typename" as the minimal valid subfield because that is always available and doesn't require a deep dive into the schema. Fields returning scalar types don't require subfields. The function returns a list of tuples in the format `(field_name, subfields)`.

- **`measure_field_timings(url: str, fields: list)` → `list`:** This function takes a GraphQL endpoint URL and a list of fields (each with optional subfields), constructs GraphQL queries for each field, and measures the time it takes to get a response. It uses *send_query* to perform the actual request and records the elapsed time for each field. If a request fails, it logs a warning with the HTTP status code. Finally, it returns a list of tuples in the form `(field_name, subfields, elapsed_time)`, sorted by the response time in descending order (slowest queries first).

- **`send_batched_queries(url: str, field: str, batch_size: int, subfield: str = None) → None:`** This function sends a batch of identical GraphQL queries to a given endpoint. It constructs a query for the given field, optionally including a subfield if provided. The same query is then duplicated *batch_size* times and sent as a batch using the *send_query* function. If the request succeeds, it prints a success message along with the total time taken. If the request fails, it prints the HTTP error code and the response content.

- **`main():`** Depending on the arguments given when running the script it loads loads the schema or does introspection, then extracts all possible queries from the schema, sends them, measures the response time and constructs and sends a batching query with the query that had the longest response time (i.e. was probably most resource intensive).

In the same folder there is also a file called `DoSBatchingWithTokens.py`, where the send HTTP POST includes tokens (one can get the tokens by e.g. inspecting the HTTP packets in BurpSuite and then coping them into the script). However, we have not found requests with tokens to be more useful, because all the projects that require tokens for (some) queries have batching disabled.

## 4.3 Results

We ran the DoS Batching fuzzer against six GraphQL deployments: DVGA, Vendure, Saleor, Keystone, Strapi, and the Trevorblades Public Endpoint. All results from these tests are stored under `DoS_Batching/` (as pictures to see the run command).

Below is a summary of our findings:

1. **Keystone, Strapi, Vendure (both GraphQL endpoints) & Trevorblades:**
   For these projects the fuzzer discovered that the servers reject any batched GraphQL requests. In each case, the HTTP response contained an explicit error message such as `"Operation batching disabled"`. Therefore, a DoS attack through query batching is impossible for these projects. Since, we ran all projects with their default settings, this implies that these large, production-grade projects have already disabled GraphQL batching in their default configurations, preventing this style of DoS outright.

2. **DVGA (Damn Vulnerable GraphQL Application):**
   In "Expert Mode", DVGA applies a Cost Query Analysis, which prevents the batching of resource intensive queries like system updates. Consequently, any batched request containing high-cost queries should be rejected before execution. In "Standard Mode", however, DVGA allows batching. Our fuzzer identified the single slowest query and

successfully sent batches. The sizes we ran the fuzzer for are quite small ($3$ and $5$) because the time consumed for these queries is already very high (one query takes at least $20$ seconds, sometimes up to $50$). Therefore, without cost analysis enabled, DVGA is vulnerable to DoS through batching.

3. **Saleor:**
   Saleor's default deployment does not disable batching, as all our batching queries went through. However, all their queries execute extremely quickly with the slowest field taking around $0,03$ seconds. Because individual queries finish so rapidly, we had to use very large batch sizes ($1000$ and $10000$) before observing response times of over $1$ second. Even for a batch of $10.000$ the response only took $\approx 10$ seconds. Even so, this means that Saleor is susceptible to batching-based DoS in default configuration, simply because it has no per-request complexity limit and allows multiple operations per HTTP request.

**Things to note:** In several cases, the queries build by the script (to measure the time of each field) return HTTP Errors. There are two main reasons for this:

- The automated fuzzer has no valid authentication tokens and, thus, when it attemps to query protected fields we get an error.
- Certain fields (e.g. `user(where:{id:"abc"})`) require specific argument values (in the example `"where:{id:"abc"}"`), causing errors like `"Argument 'id' of required type ID!' was not provided."`.

This is the case because automatically including such additions would have made the script overly complicated.
Although these errors prevent the fuzzer from fully enumerating every field, they do not affect the ability to detect whether batching is allowed or disabled. In other words, even with partial schema coverage, we still accurately determine batching support, vulnerability status and possible attack fields (though there may be some with higher response times that were missed because of errors).

**Overall Takeaways:**

- Large, actively maintained GraphQL projects (Keystone, Strapi, Vendure) typically ship with batching disabled or guarded by cost analysis, mitigating this DoS class by default.
- Default configurations of DVGA Standard Mode and Saleor illustrate that enabling batching without cost limits can lead to severe performance degradation or service unavailability.

- Automated fuzzers will sometimes hit permission or argument errors; despite this, identifying batching support remains straightforward.

## 4.4 Mitigation

Several strategies exist to prevent or limit successful DoS attacks that abuse GraphQL batching (two of them can be seen from section 4.3):

1. **Disable Batching Entirely**
   Most GraphQL server frameworks (Apollo Server, Express-GraphQL, Prisma, etc.) provide an option to disable multiple operations per HTTP request. This completely eliminates the risk of batching-based DoS, as clients cannot submit more than one operation at a time. However, it loses the performance benefit of reduced HTTP overhead and parallelized data fetching, especially for mobile or high-latency clients.

2. **Enforce Query Cost or Complexity Analysis**
   Introduce middleware that inspects each incoming query's shape, estimates a "cost" based on field depth and cardinality, and rejects any request above a configured threshold. This allows batching but caps total resource consumption, so even if a client submits $100$ identical queries in one batch, the cumulative cost is assessed and the request is rejected if it exceeds limits. However, this requires careful tuning of cost weights (per-field, per-relationship) and complexity calculators may occasionally misjudge or be bypassed by novel query shapes.

3. **Result Caching or Query Persisting**
   Lastly, if the server caches each unique query's result (e.g. via Redis or an in-memory cache), then multiple identical batched requests will hit the cache after the first execution. This greatly reduces resource consumption. However, there are limitations here as well – mainly that it's only effective when all queries in the batch are identical. If each query has different arguments, caching may not be effective.

All of these strategies (disabling batching, caching, cost analysis) effectively mitigate, drastically reduce or eliminate the risk of DoS attacks that rely on GraphQL's batching feature.

# 5 DoS Recursion

## 5.1 Concept

Recursion-based Denial of Service attacks exploit the fact that GraphQL schemas can include fields that reference themselves (directly or indirectly) or each other (e.g. user has products and products belong to users), allowing deeply nested queries that consume unbounded server resources. A malicious client can craft a query that repeatedly expands a recursive field – causing excessive CPU usage, memory exhaustion, or stack overflows – leading to service degradation or outage.

Since automatically detecting arbitrary recursion points in an arbitrary schema via introspection is non-trivial, our approach divides the task into two parts:

- **Manual Recursion Specification:** Examine the visualized schema (e.g. via GraphQL Voyager) to identify a recursive field chain. Then build a valid query based on the found recursion.
- **Recursive Query Execution:** Here we have two scripts.
  The first one `DoS_Recursion.py` is very basic and accepts a JSON file containing the entire recursion query and the GraphQL endpoint URL. It then uses those to send a request. The functions here are trivial, therefore, in the following we will mainly explain the second file.
  The second file, `DoS_RecursionAutomatic.py`, needs the GraphQL endpoint and the recursion chain, e.g. if the recursion is made of "users{product{users ...}}", it needs "users product" as input. There are several more optional inputs, which we will explain in the next section. The basic idea for the script is that it iteratively increases the nesting depth:
  1. Construct a GraphQL query by repeating the recursive field chain `n` times.
  2. Send the query and measure response time and status.
  3. Repeat for increasing values of `n` until the server responds with an error, times out, or response takes too long (i.e. server overwhelmed).

By plotting response time (or observing failures) against depth, we can determine the maximum safe nesting level and identify whether the endpoint is vulnerable to recursion-based DoS. This semi-automated method balances ease of use with the practical difficulty of fully automating recursion discovery.

## 5.2 Code Documentation

The DoS recursion functionality is implemented in `DoS_RecursionAutomatic.py`. Since fully automating the discovery of recursive fields was too complicated, the recursion path has to be identified manually and specified when calling the script. The script accepts the following command-line arguments:

- **URL of the GraphQL endpoint** (`-u` or `--url`)
- **Fields for recursion chain** (`-c` or `--chain`)
  A space-separated list of the nested fields that form the recursion. *Example*: for queries like `"users{product{users{...}}}"` you would pass `"-c users product"`.
- **Optional root field** (`--root`) and its arguments (`--root-args`)
  If the very first field of the query is different from the main recursion path. Then one has to specify that first field with these tags and the main recursion will be wrapped with these arguments. *Example*: a query is `"user(id: "abc") {...}"` then call the function with `"--root user --root-args "{id: "abc"}"`.
- **Tuning parameters**
  - `--start-depth` (default: 2) – the initial nesting depth.
  - `--max-depth` (default: 20) – the maximum depth to test.
  - `--step` (default: 2) – the increment between depths.

  Note: `start-depth` and `step` must both be even, to align with the reverse-indexing logic in the `build_recursive_query` function.

The functions in `DoS_Recursion.py` and `DoS_RecursionAutomatic.py` are:

- **`load_json_file(filepath)`:** Loads the JSON file containing the recursion query data (only `DoS_Recursion.py`).
- **`send_graphql_request(url, payload)`:** Sends an HTTP POST to the specified GraphQL endpoint with the given payload and measures the elapsed time. Afterwards, it prints the success or failure and the response. The returned values differ slightly between the two scripts.

- **`build_recursive_query(chain, depth, root_field=None, root_args=None):`** Dynamically constructs the nested GraphQL query string. It initializes `nested` with `"__typename"` (because at the end of the recursion we have to ask for some data) as the leaf selection. Then loops `depth` times, each time prepending one level of nesting (therefore we build the query from inside out) so that the first element in the given `chain` becomes the outermost field. Lastly, if root arguments were given it wraps the final nested block with those. Returned is the full `"query recursiveTest { ... }"` string.

- **`main():`** The main function parses and validates the given arguments and calls the relevant functions.
  In `DoS_Recursion.py` those are `load_json_file` and then the `send_graphql_request` functions.
  In `DoS_RecursionAutomatic.py` we iterate the depths from `start-depth` to `max-depth` in steps of the provided `step` and do the following for each iteration:

  - Builds the query via `build_recursive_query`.
  - Sends it with `send_graphql_request`.
  - Collects results and stops early if a request fails or its latency exceeds a threshold.

  Afterwards, it prints a final summary of which depths succeeded or failed and their timings.

## 5.3 Results

We ran the DoS Recursion fuzzer against the same six GraphQL deployments used for the batching tests: DVGA, Vendure, Saleor, Keystone, Strapi, and Trevorblades Public Endpoint. All results from these tests are stored under `DoS_Recursion/`. Most of the results are for the `DoS_RecursionAutomatic.py` script. For the other script `DoS_Recursion` we only saved the interesting results as that script simply sends an HTTP Post with the provided JSON and prints the answer it got.

Below is a summary of our findings for each project:

**DVGA (Damn Vulnerable GraphQL Application):**
Here we used the `DoS_RecursionAutomatic.py` script with a depth increase of 2 for each query (e.g. adding one more `{pastes{user}}`). All the requests went through and the response time doubled with each increase in depth for depths $> 10$ (exponential growth with depth). For depth 20 it already took $2,44$ seconds.

To assess the robustness further, we tested even deeper recursion levels and observed that the server does not enforce any recursion depth limits or protective measures against Denial of Service (DoS) via deeply nested queries. Despite the heavy load, the server did not crash. This is likely because DVGA runs inside a Docker container, which by default has access to a substantial portion of the host's memory. During our test, the container's memory usage exceeded $2$ GB, after which we manually restarted it. It's reasonable to conclude that had the test continued, the server might eventually have failed due to memory exhaustion.

**Vendure**

For this project we only tested the `/shop` endpoint, because the `/admin` endpoint required authentication and complex arguments, which was too time consuming to build a valid query for (additionally the results should be very similar to the results we got from the shop endpoint). The `DoS_RecursionAutomatic.py` script timed out at $10$ seconds for depths beyond $12$, which means that the response took longer than $10$ seconds and thus the script stopped waiting for a response from the server. Running the `DoS_Recursion.py` script with a manually created long JSON payload (`RecursionVendureShopLong.json`), resulted in the server ultimately crashing with a JavaScript heap out-of-memory error (the result is stored in the file `ResultsRecursionVendureShopServerCrash.png`). This is a perfect example of what recursion attacks can achieve and demonstrates that Vendure's runtime cannot handle deeply nested objects and will exhaust available memory. The implication of this is that Vendure's default configuration lacks a recursion depth limit, so a sufficiently deep recursive query can crash the server process.

**Saleor**

We attempted to construct a valid recursion chain based on the schema (e.g. `products {category {products {...} } }` ); however, as shown in `ResultsRecursionSaleorEmpty.png`, the field that ought to contain a non-empty nested result of category $\rightarrow$ products $\rightarrow$ category (according to the the graphical interface and BurpSuite, products of the specified category exist) returned an empty array. Therefore, although the schema permitted nesting, actual query results returned empty lists for each level and no errors were thrown, but no data was returned either. As a result, deep recursion queries did nothing because we cannot call something recursively if it is empty. We could not observe any latency or server error due to the absence of nested data. In conclusion, Saleor's default dataset did not have populated nested fields for recursion testing, so no meaningful recursion results were obtained.

**Keystone**

Using the `DoS_RecursionAutomatic.py` script, *all* depths from $2$ up to $702$ returned HTTP 200 with correct and valid responses. Response times grew very slowly: approximately $0,01$ seconds at depth $2$, up to approximately $0,16$ seconds at depth $702$. Even at depth $702$, Keystone successfully resolved and serialized nested objects. At depth $802$, the server responded

with HTTP 500 and a GraphQL error "Maximum call stack size exceeded", originating from Node's `JSON.stringify(...)`.
The reason for this should be that Keystone 6+ uses Prisma to talk to the database and DataLoader to cache any records it has already seen in a single request (= previous database lookups). Since each post's author (our query is a recursion on author $\rightarrow$ posts $\rightarrow$ author, with the author always being the same) was the same user in our test data, deeper recursion hit cached results rather than issuing new SQL queries. Therefore, database work became effectively $O(1)$ after depth $2$, and only the cost of turning the nested result into JSON increased with depth (JSON serialization). When the nesting reached around depth $802$, the JavaScript engine couldn't finish serializing the object and it failed (overflow). Thus, while Keystone can handle extremely deep recursion without database load, its JSON serializer eventually overflows. This results in a stack overflow error and a 500 response but does not crash the entire server.

**Strapi**

Similar to Vendure, a manual long JSON payload caused Strapi to run out of heap memory, crashing the Node.js process. The `DoS_RecursionAutomatic.py` script stopped sending queries when response times exceeded the scripts $5$ second threshold at relatively low depths (around $14$), indicating that incremental serialization cost combined with large payload size made each request too slow (response times doubled starting from around depth $8$). In conlusion, Strapi's default setup does not impose a recursion limit or use optimized caching for nested relationships, so deep nesting can exhaust memory or cause extreme latency.

**Trevorblades Public Endpoint**

This public endpoint enforces a maximum query depth of approximately $8$. Any recursion query deeper than $8$ returned a GraphQL error "Query depth limit exceeded". Below depth 8, response times remained low (around $0,3$ seconds), showing no significant latency increases until the depth limit was reached. Therefore, Trevorblades has a built-in depth limit that effectively prevents recursion-based DoS, while still allowing moderate nested queries.

**Overall Takeaway**

Many large, actively maintained GraphQL projects do not enable protections against DoS attacks via deeply nested queries by default. While several of the tested frameworks offer an option to set a maximum query depth, those safeguards must be explicitly configured rather than coming enabled out of the box.

## 5.4 Mitigation

To prevent or limit successful recursion-based DoS attacks, there are many stategies, here are a few:

1. **Depth Limiting or Complexity Limiting**: Enforce a maximum allowed nesting depth or compute a complexity score per operation. This immediately stops deeply nested queries that would otherwise consume excessive CPU or memory. However, it may break legitimate use cases requiring deep relationships and requires careful tuning of allowed depth or complexity threshold.

2. **Relation Lookup Caching (Dataloader Pattern)**: Like Keystone did, we can use a batching or caching layer (e.g. Prisma's Dataloader for Node.js) so that resolving nested relationships beyond the first few levels hits an in-memory cache rather than repeatedly querying the database. Through this, the database load can be reduced to O(1) after initial lookups. Additionally, it minimizes repeated fetches for cyclic or shared references. The disadvantage is that it does not prevent JSON serialization overhead and deeply nested objects may still exhaust memory.

3. **Serialization Safeguards**: Configure the GraphQL server's JSON serializer or HTTP framework to impose a serialization depth or size limit (e.g. custom middleware that aborts if object nesting exceeds a threshold). The advantage is that this prevents runtime stack overflows (as in Keystone) or heap exhaustion (as in Strapi/Vendure). However, it adds overhead to serialization logic and legitimate large-but-finite nested responses may be rejected if limits are too strict.

4. **Rate Limiting or Throttling**: Lastly, one can impose per-client or per-IP rate limits on GraphQL HTTP requests. This would slow down repeated deep recursion attempts and allow caches or defenses time to clear but legitimate batch or high-frequency use cases may require higher quotas or whitelisting.

## Additions

We originally intended to include more GraphQL projects, but identifying suitable candidates proved challenging – as we had to rely on project descriptions that explicitly mentioned GraphQL (GraphQL is only an API not a language/... thus filtering for it is hard). Moreover, many of the repositories we discovered were primarily testing or demo projects, while we sought real-world applications to obtain realistic results. In addition, a number of these projects would not run out of the box due to missing dependencies, misconfigurations, or incomplete documentation. Finally, some candidates required extensive setup, including database development and configuration, which exceeded our available time. Consequently, we were limited to the six projects used for testing.

## Contribution

Tomás Sobral Teixeira did literature review, wrote the Introduction and SQL exploit and tested possible GitHub projects to run the implementations on.

Eva Sophia Schreyer did literature review, wrote the Reconnaisscance, DoS Batching and Recursion sections and researched schema enumeration and reconnaissance when introspection is disabled. Furthermore, she researched and tested possible GitHub projects to test the exploits on.